

Hands-on Assignment 2

2466025 박혜린

2-1. Longest increasing subsequene

Step1. Prompt AI to generate code

Prompt

Find the LIS for the array [10,22,9,33,21,50,41,60] and print the actual elements of that sequence not just the length. Generate code in Python

Finding Longest Increasing Subsequence



Find the LIS for the array [10,22,9,33,21,50,41,60] and print the actual elements of that sequence not just the length. Generate code in Python



The **Longest Increasing Subsequence (LIS)** for the array [10, 22, 9, 33, 21, 50, 41, 60] is a sequence where each element is strictly greater than the previous one, and its length is maximized.

The Result

For your specific input, the LIS has a **length of 5**. While multiple sequences can sometimes exist, the standard result for this array is:

[10, 22, 33, 50, 60] (or alternatively [10, 22, 33, 41, 60]).

Python Implementation

To find both the length and the actual elements, we use **Dynamic Programming**. We maintain an array `tails` to track the smallest possible tail for all increasing subsequences of various lengths, and we store the "predecessors" to reconstruct the path at the end.

Python

```
def find_lis(nums):
    if not nums:
        return []

    n = len(nums)
    # dp[i] stores the length of LIS ending at index i
    dp = [1] * n
    # parent[i] stores the index of the previous element in the LIS
    parent = [-1] * n

    for i in range(n):
        for j in range(i):
            if nums[i] > nums[j] and dp[i] < dp[j] + 1:
                dp[i] = dp[j] + 1
                parent[i] = j

    # Find the index of the maximum element in dp
    max_len = max(dp)
    current_idx = dp.index(max_len)

    # Reconstruct the sequence
    lis_sequence = []
    while current_idx != -1:
        lis_sequence.append(nums[current_idx])
        current_idx = parent[current_idx]

    return lis_sequence[::-1]

# Input Array
arr = [10, 22, 9, 33, 21, 50, 41, 60]
result = find_lis(arr)

print(f"Input Array: {arr}")
print(f"LIS Length: {len(result)}")
print(f"LIS Elements: {result}")
```

How it works

1. **Comparison:** We compare every element `nums[i]` with all previous elements `nums[j]`.
2. **State Update:** If `nums[i] > nums[j]`, it means `nums[i]` can extend the subsequence ending at `j`.
3. **Backtracking:** By keeping a `parent` array, we don't just know how long the sequence is; we keep a "breadcrumb trail" that allows us to jump backward from the end of the sequence to the beginning.

) Gemini 프롬프트 입력

도구

빠른 모델

AI generated code

```

def find_lis(nums):
    if not nums:
        return []

    n = len(nums)
    # dp[i] stores the length of LIS ending at index i
    dp = [1] * n
    # parent[i] stores the index of the previous element in
    the LIS
    parent = [-1] * n

    for i in range(n):
        for j in range(i):
            if nums[i] > nums[j] and dp[i] < dp[j] + 1:
                dp[i] = dp[j] + 1
                parent[i] = j

    # Find the index of the maximum element in dp
    max_len = max(dp)
    current_idx = dp.index(max_len)

    # Reconstruct the sequence
    lis_sequence = []
    while current_idx != -1:
        lis_sequence.append(nums[current_idx])
        current_idx = parent[current_idx]

    return lis_sequence[::-1]

# Input Array
arr = [10, 22, 9, 33, 21, 50, 41, 60]
result = find_lis(arr)

print(f"Input Array: {arr}")
print(f"LIS Length: {len(result)}")
print(f"LIS Elements: {result}")

```

```

parkchaerin@bagchaelins-MacBook-Air ~ % /usr/local/bin/python3 /Users/parkchaerin/ha2_1.py
Input Array: [10, 22, 9, 33, 21, 50, 41, 60]
LIS Length: 5
LIS Elements: [10, 22, 33, 50, 60]

```

Step2. Analyze the space complexity of the AI's initial code.

1. Input storage

`nums` is passed by reference, so **O(1)** extra space.

2. DP arrays

Both `dp` and `parent` are initialized to length `n`, contributing **O(n) + O(n) = O(n)**.

3. Reconstruction

`lis_sequence` grows up to length `n` in the worst case contributing **O(n)**.

4. Scalar variables

`n`, `max_len`, `current_idx` are all **O(1)**.

Therefore, Overall Space Complexity is **O(n)**.

Step 3. Prompt AI again for optimization

storing entire arrays for every state

```

dp[i] = [10, 22, 33, 50, 60] # storing the full subsequenc
e at each index

```

This would cost **O(n²)** space, since each of the `n` states holds a list of up to `n` elements.

However, in AI generated code, it uses a **smart reconstruction trick**. `parent` array acts like a linked list of back-pointers.

```

index:  0   1   2   3   4   5   6   7
nums:  [10, 22,  9, 33, 21, 50, 41, 60]
dp:     [ 1,  2,  1,  3,  2,  4,  4,  5]
parent: [-1,  0, -1,  1,  0,  3,  3,  5]

```

At the end, it traces back through parent pointers to reconstruct the sequence . Therefore, you only need one integer per state, not a full array.

Therefore, Step3 is omitted because no additional prompting process was needed to point out memory waste and require optimization.

Step 4. Final optimized code

AI generated code's time complexity is $O(n^2)$ due to nested loop. Therefore, I add binary search function so that I can replace the nested loop with a single loop and make time complexity into $O(n\log n)$.

```
### generated binary_search function manually

# Binary search to find the leftmost position where target
# can be inserted
# to maintain sorted order in tails array
def binary_search(tails, target):
    left, right = 0, len(tails) - 1
    while left <= right:
        mid = (left + right) // 2
        if tails[mid] < target:
            left = mid + 1 # target is larger, search right half
        else:
            right = mid - 1 # target is smaller or equal, search left half
    return left # return insertion position

###

def find_lis_optimized(arr):
    n = len(arr)
    if n == 0:
        return 0, []
    ### generated manually
    tails = [] # stores the smallest tail element for each LIS length
    indices = [] # stores the actual arr index corresponding to each tails entry
```

```

    parent = [-1] * n # parent[i] = index of previous element in LIS ending at i

    for i, num in enumerate(arr):
        pos = binary_search(tails, num) # binary search로 nested loop 대체

        if pos == len(tails):
            # num is larger than all tails → extends the longest subsequence
            tails.append(num)
            indices.append(i)
        else:
            # num replaces an existing tail → keeps tails values as small as possible
            tails[pos] = num
            indices[pos] = i

            # link current element to the previous element in the LIS
            if pos > 0:
                parent[i] = indices[pos - 1]
            ###

    max_len = len(tails) # length of LIS = number of entries in tails (no need for max())
    current_idx = indices[-1] # the last entry in indices always points to the end of the longest subsequence
    sequence = [] # backtrack through parent pointers to reconstruct the actual LIS
    while current_idx != -1:
        sequence.append(arr[current_idx])
        current_idx = parent[current_idx]

    return max_len, sequence[::-1] # sequence is built in reverse order, so flip it

nums = [10, 22, 9, 33, 21, 50, 41, 60]

```

```
lis_length, lis_sequence = find_lis_optimized(nums)
print(f"Input Array: {nums}")
print(f"Longest Increasing Subsequence: {lis_sequence}")
print(f"Length of LIS: {lis_length}")
```

```
● parkchaerin@bagchaelins-MacBook-Air ~ % /usr/local/bin/python3 /Users/parkchaerin/ha2_1.py
Input Array: [10, 22, 9, 33, 21, 50, 41, 60]
Longest Increasing Subsequence: [10, 22, 33, 41, 60]
Length of LIS: 5
```

The parent array records the index of the previous element whenever $pos > 0$ is satisfied during the single loop traversal. Once the loop finishes, `indices[-1]` is used as the starting point since it always points to the last element of the longest subsequence. The algorithm then follows the chain stored in `parent`, appending `arr[current_idx]` at each step until `current_idx` reaches `-1`, which means the beginning of the subsequence has been reached. Since elements are collected from end to start during this traversal, `sequence[::-1]` is applied at the end to restore the correct LIS order.

2-2. Minimum number of US postage stamps

Step 1. Prompt AI

Prompt

We have nine postage stamps and each costs, 1, 10, 21, 34, 70, 100, 350, 1225, 1500 cents. Implement a Dynamic Programming solution to find the minimum count of postage stamps for the 140-cent target.

We have nine postage stamps and each costs, 1, 10, 21, 34, 70, 100, 350, 1225, 1500 cents. Implement a Dynamic Programming solution to find the minimum count of postage stamps for the 140-cent target.



To find the minimum number of stamps required to reach a target value, we can use the **Change-Making Problem** approach via Dynamic Programming (DP).

The core idea is to build a table where each entry $dp[i]$ represents the minimum stamps needed to make the value i . We build this up from 1 to the target (140).

The Logic

For every stamp s and every value i , if $s \leq i$, the minimum stamps for i is:

$$dp[i] = \min(dp[i], dp[i - s] + 1)$$

Python Implementation

Python



```
def min_stamps(stamps, target):
    # Initialize DP table with a value larger than any possible solution
    dp = [float('inf')] * (target + 1)

    # Base case: 0 cents requires 0 stamps
    dp[0] = 0

    # Iterate through all values from 1 to target
    for i in range(1, target + 1):
        for stamp in stamps:
            if stamp <= i:
                dp[i] = min(dp[i], dp[i - stamp] + 1)

    return dp[target]

# Input Data
stamp_values = [1, 10, 21, 34, 70, 100, 350, 1225, 1500]
target_value = 140

result = min_stamps(stamp_values, target_value)
print(f"Minimum stamps for {target_value} cents: {result}")
```

Step-by-Step Breakdown for 140 Cents

While the computer does the heavy lifting, here is how the DP logic arrives at the solution for your specific set:

Strategy	Stamps Used	Count
----------	-------------	-------

Step-by-Step Breakdown for 140 Cents

While the computer does the heavy lifting, here is how the DP logic arrives at the solution for your specific set:

Strategy	Stamps Used	Count
Greedy Approach	100 + 34 + 1 + 1 + 1 + 1 + 1 + 1	8
DP Optimal 1	70 + 70	2
DP Optimal 2	100 + 10 + 10 + 10 + 10	5

 Sheets로 내보내기



The Result:

The Dynamic Programming algorithm identifies that using **two 70-cent stamps** is the most efficient solution. Even though the "Greedy" method (taking the largest possible stamp first) is common for standard currency, it fails here because 140 is a perfect multiple of 70.

Minimum Count: 2

    

 Gemini 프롬프트 입력

+  도구

빠른 모델 ▾



AI generated code

```
def min_stamps(stamps, target):  
    # Initialize DP table with a value larger than any possible solution  
    dp = [float('inf')] * (target + 1)  
  
    # Base case: 0 cents requires 0 stamps  
    dp[0] = 0  
  
    # Iterate through all values from 1 to target  
    for i in range(1, target + 1):  
        for stamp in stamps:  
            if stamp <= i:  
                dp[i] = min(dp[i], dp[i - stamp] + 1)  
  
    return dp[target]
```

```
# Input Data
stamp_values = [1, 10, 21, 34, 70, 100, 350, 1225, 1500]
target_value = 140

result = min_stamps(stamp_values, target_value)
print(f"Minimum stamps for {target_value} cents: {result}")
```

```
● parkchaerin@bagchaelins-MacBook-Air ~ % /usr/local/bin/python3 /Users/parkchaerin/ha2_2.py
Minimum stamps for 140 cents: 2
```

Step 2. Print the Dynamic Programming table

```
def min_stamps(stamps, target):
    # Initialize DP table with infinity for all values (meaning "not yet reachable")
    dp = [float('inf')] * (target + 1)

    # Base case: 0 cents requires 0 stamps
    dp[0] = 0

    # Fill the DP table from 1 cent up to the target
    for i in range(1, target + 1):
        # Try every stamp denomination
        for stamp in stamps:
            # Only use this stamp if it doesn't exceed the current amount
            if stamp <= i:
                # If using this stamp gives a better (smaller) count, update dp[i]
                dp[i] = min(dp[i], dp[i - stamp] + 1)

    ### generated manually
    # Print the DP table header
    print(f"{'Target':>8} | {'Min Stamps':>10}")
    print("-" * 22)

    # Print each row: target amount and the minimum stamps
```

```

needed to reach it
    for i in range(1, target + 1):
        # Show "N/A" if the amount is unreachable with the
        # given stamp denominations
        val = dp[i] if dp[i] != float('inf') else "N/A"
        print(f"{i:>8} | {str(val):>10}")
    ###

    # Return the minimum number of stamps needed for the ta
    # rget amount
    return dp[target]

# Input Data: available stamp denominations (in cents)
stamp_values = [1, 10, 21, 34, 70, 100, 350, 1225, 1500]

# The target amount we want to reach (in cents)
target_value = 140

# Run the function and store the result
result = min_stamps(stamp_values, target_value)

# Print the final answer
print(f"\nMinimum stamps for {target_value} cents: {result}")

```

```
parkchaerin@bagchaelins-MacBook-Air ~ % /usr
/local/bin/python3 /Users/parkchaerin/ha2_2.
py
Target | Min Stamps
-----|-----
1 | 1
2 | 2
3 | 3
4 | 4
5 | 5
6 | 6
7 | 7
8 | 8
9 | 9
10 | 1
11 | 2
12 | 3
13 | 4
14 | 5
15 | 6
16 | 7
17 | 8
18 | 9
19 | 10
20 | 2
21 | 1
22 | 2
23 | 3
24 | 4
25 | 5
26 | 6
27 | 7
28 | 8
29 | 9
30 | 3
31 | 2
32 | 3
```

```
34 | 1
35 | 2
36 | 3
37 | 4
38 | 5
39 | 6
40 | 4
41 | 3
42 | 2
43 | 3
44 | 2
45 | 3
46 | 4
47 | 5
48 | 6
49 | 7
50 | 5
51 | 4
52 | 3
53 | 4
54 | 3
55 | 2
56 | 3
57 | 4
58 | 5
59 | 6
60 | 6
61 | 5
62 | 4
63 | 3
64 | 4
```

65	3
66	4
67	5
68	2
69	3
70	1
71	2
72	3
73	4
74	5
75	4
76	3
77	4
78	3
79	4
80	2
81	3
82	4
83	5
84	4
85	5
86	4
87	5
88	4
89	3
90	3
91	2
92	3
93	4
94	5
95	6
96	5
97	4
98	5
99	4
100	1
101	2
102	3
103	4
104	2
105	3
106	4
107	5
108	6
109	5
110	2
111	3
112	3
113	4
114	3
115	4
116	5
117	6
118	5

119		6
120		3
121		2
122		3
123		4
124		4
125		3
126		4
127		5
128		6
129		7
130		4
131		3
132		4
133		4
134		2
135		3
136		4
137		5
138		3
139		4
140		2

Minimum stamps for 140 cents: 2

This divergence happens at 30 cents because 30 is the smallest value where greedily picking the largest stamp (21cents) leaves a remainder of 9cents that no denomination other than 1 cents can cover, forcing 9 extra stamps. DP avoids this by already knowing $dp[10]=1$ and $dp[20]=2$, so it simply uses three 10 cents stamps instead. Since 21 is not a multiple of 10, any target where greedy picks 21 cents risks creating an inefficient remainder. therefore , 30 is the first case.

Step 3. Test the robustness of the AI's algorithm

Target: 7 cents

DP

```
def min_stamps(stamps, target):
    dp = [float('inf')] * (target + 1)
    dp[0] = 0

    for i in range(1, target + 1):
        for stamp in stamps:
            if stamp <= i:
```



```

# Key: stamp value, Value: number of stamps used
used_stamps = {}

# Iterate through each stamp denomination (assumed to be sorted in descending order)
for stamp in denominations:
    if target_cost == 0: # If no cost remains, stop early (optimization)
        break

    # Calculate how many stamps of this value fit into the remaining cost
    num_stamps = target_cost // stamp

    if num_stamps > 0: # If at least one stamp can be used
        count += num_stamps # Add to total count
        used_stamps[stamp] = num_stamps # Record usage in dictionary
        # Update the remaining cost
        target_cost %= stamp

    return count, used_stamps # Return total number of stamps and detailed breakdown

# Input Data
cost = 7
stamps = [5, 4, 1]

# Execution
total_count, breakdown = minimize_stamps(cost, stamps)

print(f"Total stamps needed: {total_count}")
print("Breakdown of stamps used:")
for val, qty in breakdown.items():
    print(f"- {qty} stamp(s) of {val} cents")

```

```

PS C:\Users\cse\AppData\Local\Programs\Microsoft VS Code> & C:\Users\cse\AppData\Local\Programs\Microsoft VS Code\python\python.exe C:\Users\cse\AppData\Local\Programs\Microsoft VS Code\python\python.exe top/Untitled-1.py
Total stamps needed: 3
Breakdown of stamps used:
- 1 stamp(s) of 5 cents
- 2 stamp(s) of 1 cents

```

Both DP and Greedy requires minimum 3 stamps.

Target: 8 cents

DP

```

def min_stamps(stamps, target):
    dp = [float('inf')] * (target + 1)
    dp[0] = 0

    for i in range(1, target + 1):
        for stamp in stamps:
            if stamp <= i:
                dp[i] = min(dp[i], dp[i - stamp] + 1)

    # Print the DP table
    print(f"{'Target':>8} | {'Min Stamps':>10}")
    print("-" * 22)
    for i in range(1, target + 1):
        val = dp[i] if dp[i] != float('inf') else "N/A"
        print(f"{i:>8} | {str(val):>10}")

    return dp[target]

# Input Data
stamp_values = [5,4,1]
target_value = 8

result = min_stamps(stamp_values, target_value)
print(f"\nMinimum stamps for {target_value} cents: {result}")

```



```

        target_cost %= stamp

    return count, used_stamps # Return total number of stamps and detailed breakdown

# Input Data
cost = 8
stamps = [5, 4, 1]

# Execution
total_count, breakdown = minimize_stamps(cost, stamps)

print(f"Total stamps needed: {total_count}")
print("Breakdown of stamps used:")
for val, qty in breakdown.items():
    print(f"- {qty} stamp(s) of {val} cents")

```

```

PS C:\Users\cse\AppData\Local\Programs\Microsoft VS Code> & C:\Users\cse\AppData\Local\Programs\Microsoft VS Code> python top/Untitled-1.py
Total stamps needed: 4
Breakdown of stamps used:
- 1 stamp(s) of 5 cents
- 3 stamp(s) of 1 cents

```

DP successfully output the result 2. This indicates that AI generated code is not **hardcoded** for specific inputs. Instead, it correctly evaluates all possible combinations to find the global minimum.

Step 4. Worst-case time and space complexities and discuss what structural property allow dynamic programming to succeed across all test cases where Greedy fails.

Time and Space complexity

n: the number of stamps, T: Target cost

Time complexity: $O(n \times T)$

The outer loop runs for each denomination (n iterations), and the inner loop runs from the denomination value up to Target cost (T). In the inner loop, the basic operation is $dp[i] = \min(dp[i], dp[i - stamp] + 1)$. This operation is performed up to $n \times T$. Therefore, time complexity is $O(n \times T)$.

Space complexity: $O(T)$

A DP table to store the minimum number of stamps for every value from 0 to T is needed. Regardless of how many denominations you have, the size of the array is determined by the Target cost. Therefore, space complexity is $O(T)$.

Why DP succeeds where Greedy fails

Overlapping sub-problems

DP breaks the problem down into smaller sub-problems and stores the results on the table. To make 8 cents, the algorithm first calculates and stores the optimal solutions for making 7, 4, and 3 cents. When it reaches $dp[8]$, it reuses the pre-computed result of $dp[4]$. Therefore, this algorithm avoids repetitive work and efficiently identifies the overall optimal solution for the 8-cents.

Optimal substructure

This property indicates that the optimal solution to the overall problem contains within it the optimal solutions to its sub-problems. When target cost is 8 cents, the optimal solution is two of 4. The optimal solution for 4 cents is one 4. DP identifies this relationship correctly. However, Greedy assumes that choosing the local optimum (the largest stamp, 5) will lead to the global optimum. This local optimum blocks the path to the overall optimal solution.

2-3. All-Pair-Shortest-Path Problem

Step 1. Change/add codes

```
INF = int(1e9 + 7)

def floyd_warshall(dist, length):
    ### added code
```

```

# Initialize 'next' matrix for path reconstruction
next_node = [[None] * length for _ in range(length)]

# If there's a direct edge, the next hop from p to q is
q itself
for p in range(length):
    for q in range(length):
        if dist[p][q] != INF and p != q:
            next_node[p][q] = q # reset next_node
    ###

# Main Floyd-Warshall loop – modified min to also track
path
for r in range(length):
    for p in range(length):
        for q in range(length):

            ### modified code
            if dist[p][r] + dist[r][q] < dist[p][q]:
                # routing r is
                dist[p][q] = dist[p][r] + dist[r][q]
                next_node[p][q] = next_node[p][r]
            ###

print_dist(dist, length)
return next_node

### added code
def shortest_path(next_node, start, dst):
    # Reconstruct the full path from start to dst.
    if next_node[start][dst] is None:
        return [] # No path exists
    path = [start]
    while start != dst:
        start = next_node[start][dst]
        path.append(start)
    return path
###

```

```
def print_dist(dist, length):
    for p in range(length):
        temp = ""
        for q in range(length):
            if dist[p][q] == INF:
                temp += "INF "
            else:
                temp += str(dist[p][q]) + " "
        print(temp)
```

Step 2. Use input

```
W = [
    [0, 3, INF, 7],
    [8, 0, 2, INF],
    [5, INF, 0, 1],
    [2, INF, INF, 0]
]

length = 4
next_node = floyd_warshall(W, length)
```

Step 3. Print out all the vertices on two shortest paths.

```
INF = int(1e9 + 7)

def floyd_warshall(dist, length):
    # Initialize 'next' matrix for path reconstruction
    next_node = [[None] * length for _ in range(length)]

    # If there's a direct edge, the next hop from p to q is q itself
    for p in range(length):
        for q in range(length):
```

```

        if dist[p][q] != INF and p != q:
            next_node[p][q] = q

    # Main Floyd-Warshall loop – modified min to also track
    path
    for r in range(length):
        for p in range(length):
            for q in range(length):
                if dist[p][r] + dist[r][q] < dist[p][q]:
                    dist[p][q] = dist[p][r] + dist[r][q]
                    next_node[p][q] = next_node[p][r] # <-
- KEY CHANGE

    print_dist(dist, length)
    return next_node

def shortest_path(next_node, start, dst):
    # Reconstruct the full path from start to dst.
    if next_node[start][dst] is None:
        return [] # No path exists
    path = [start]
    while start != dst:
        start = next_node[start][dst]
        path.append(start)
    return path

def print_dist(dist, length):
    for p in range(length):
        temp = ""
        for q in range(length):
            if dist[p][q] == INF:
                temp += "INF "
            else:
                temp += str(dist[p][q]) + " "
        print(temp)

# Step 2
W = [

```

```

[0, 3, INF, 7 ],
[8, 0, 2, INF],
[5, INF, 0, 1 ],
[2, INF, INF, 0 ]
]

```

```
length = 4
```

```
next_node = floyd_warshall(W, length)
```

```
# Step 3
```

```
print("\nShortest path from v1 to v3:", shortest_path(next_
node, 1, 3))
```

```
print("Shortest path from v0 to v2:", shortest_path(next_no
de, 0, 2))
```

```
Minimum Stamps for 8 cents: 2
```

```
PS C:\Users\cse\AppData\Local\Programs\Microsoft VS Code> & C:\Users\
top\Untitled-1.py
```

```
0 3 5 6
```

```
5 0 2 3
```

```
3 6 0 1
```

```
2 5 7 0
```

```
Shortest path from v1 to v3: [1, 2, 3]
```

```
Shortest path from v0 to v2: [0, 1, 2]
```

```
PS C:\Users\cse\AppData\Local\Programs\Microsoft VS Code>
```